

σ : An Operating System for Solid-State Drives

Abstract

Solid-State Drives (SSDs) are embedded systems tailored for non-volatile memory. Current SSDs abstract storage through a read/write interface over a logical address space. Fundamental trade-offs impacting performance, write amplification and durability remain hard coded in SSD firmware, and largely undocumented. This black-box approach is problematic for the correctness and efficiency of data-intensive systems, whose designs are based on educated guesses. It is a hindrance for the exploration of novel SSD designs based on programmable accelerators. In this paper, we propose a new approach to SSD firmware design. We consider SSDs as programmable devices, equipped with an operating system, σ , designed for micro-second scale, that abstracts proprietary IPs, memory and general-purpose cores to provide SSD software developers with a well-defined programming model supporting a range of applications, not just the conventional flash translation layer (FTL). Above σ 's narrow waist, SSD firmware does no longer need to be proprietary. We present the abstractions provided by σ , together with its design and implementation for the (1) best-effort and (2) deterministic modes. We evaluate σ with a simple FTL on a range of different RISC-V based hardware platforms. Our experimental study shows that σ can operate at microsecond-scale with sub- μ sec overhead, and that its performance can scale to several MIOPS.

1 Introduction

Solid-State Drives (SSDs) are complex devices that provide hosts with a simple block device abstraction over multiple non-volatile memory chips. Modern SSDs are designed to operate at microsecond-scale, with ever increasing PCIe and non-volatile memory chips bandwidths. In addition, SSDs must keep up with the evolution of the NVMe standard [54] and adapt to a wide range of workload characteristics (from KV-caches to OLTP databases). In the face of such an expanding design space, SSDs are still black boxes.

SSD vendors navigate the trade-offs between scalability and flexibility with proprietary hardware and firmware components. As a result, firmware developer teams from the SSD vendors must guess the characteristics of relevant workloads, and SSD customers must guess how SSDs work. For instance, simple questions such as "How often should my system issue fsync?", "Does replication in the logical address space improve durability?" or "How should I provision SSDs to minimize cost with my workload?" cannot be answered without guessing what a given SSD model actually does (e.g., how it manages the write cache or which garbage collection algorithm it implements).

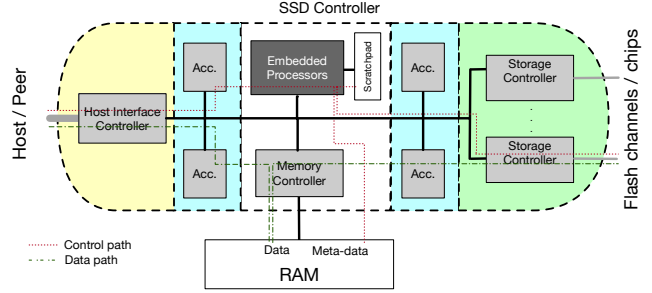


Figure 1. SSD Controller architecture.

While commercial SSDs are black boxes, their internal architecture is well documented [30, 41]. The SSD controller is the central component that manages interactions with the host (via PCIe) and with non-volatile-memory (e.g., NAND flash chips organized in channels and accessed via ONFI).

Figure 1 illustrates the main components of an SSD controller. The host interface controller abstracts NVMe management atop a PCIe controller [20, 26, 39]. The storage controller abstracts all aspects of non-volatile memory management [26, 35]. Both components incorporate DMA engines to transfer data between (i) host and SSD memory, and (ii) SSD memory and non-volatile memory. A memory controller mediates access to external RAM for data and meta-data (a few GB on modern NVMe data center SSDs), while an internal scratchpad memory is used for firmware instructions and data. Modern storage controllers combine accelerators and general-purpose processors to manage the control path (i.e., NVMe commands). Most SSDs rely on ARM cores (e.g., 5 cores 32-bit Cortex-R8 on Samsung PM9A3), while FADU relies on RISC-V cores (e.g., undocumented number of 64-bit single cores on FADU FC5161).

In this paper, we adopt this traditional architecture and focus on the nature of the SSD firmware running on embedded processors (i.e., the portion of the SSD controller with white background in Figure 1). We leave fascinating questions about alternative architectures, including programmable host-interface and storage controllers, for future work.

SSD firmware is responsible for hardware initialization, and for managing the NVMe requests workflow, which includes mapping logical to physical addresses, the so-called Flash Translation Layer (FTL), and coordinating hardware components, most notably host-interface and storage controllers [10, 26, 53]. SSD firmware must operate at microsecond-scale, it must evolve with the NVMe standard, and it must be flexible to handle diverse use cases. How can it keep up?

The approach so far, chosen by SSD vendors, is to focus on SSD controller design (e.g., for a new PCIe generation or a new form of flash chip), and develop proprietary firmware to protect the controller IP.

We consider a different approach, where SSDs are considered as programmable devices. This idea has been advanced by hyperscalers [8] and researchers [14, 19, 30, 37, 43, 44], but largely ignored by SSD vendors, until now. FADU is currently marketing a novel approach encouraging their customers to engage with them to co-design SSD controllers and customized firmware [11].

There is an obvious trade-off between the performance provided by hardware processing of I/O requests [2, 19, 20], and the flexibility provided by software. The trends towards programmability suggests a larger role given to software, in combination with hardware acceleration (on the common path) [35]. We leave programmable accelerators as a topic for future work. In this paper, we focus on making software as efficient as possible on general-purpose cores.

We posit that SSD controllers should be equipped with an operating system (OS), operating at microsecond-scale, that abstracts IP sensitive hardware components and supports a programming model for SSD applications, including but not restricted to the conventional FTL. Both SSD vendors and data-system designers should be able to develop, test and deploy SSD firmware as an embedded application.

Why consider an operating system in an embedded system such as an SSD? Legacy single-threaded SSD firmware took on many of the responsibilities of an OS such as scheduling and hardware abstraction [26], while existing work on multi-threaded and multi-core SSD firmware relies on an operating system [10, 23, 52] without exploring its impact on design or performance (See Section 2 for details).

Exploring the role of the operating system in SSDs, at microsecond-scale, thus remains an open problem. Our central hypothesis is that an OS that is well-suited for SSDs enables (i) more flexibility, (ii) more predictability, (iii) better testability, (iv) no loss of scalability, and (v) transparency over the necessary trade-offs.

This paper introduces σ , an operating system aimed at RISC-V based SSD controllers, that we designed to test this hypothesis. Our contributions are the following:

1. We characterize the design space for SSD operating systems and the position of existing systems, such as Embedded Linux and FreeRTOS, within that space. We identify the regions of the space we explore with σ , and form hypothesis to evaluate our design.
2. We design σ based on two central abstractions: region-based memory management and tagged-token dataflow of tasks. We propose a partitioned scheduling algorithm [6] and two different execution models.
3. We evaluate σ with a simple FTL application on three RISC-V systems. The experimental results show that our designs achieve sub- μ -sec overhead for task transition, a tenfold improvement over existing systems. We also show that our system achieves below 3μ sec latency and up to 2 MIOPS when replicated on 4 clusters of 4 cores.

2 Background

In this section, we survey existing work on SSD firmware and programmable SSDs. We lay particular emphasis on the role of the operating system in existing systems.

2.1 SSD Firmware.

The original SSD patent [15] introduces the SSD controller as the core of the system and mentions a narrow role for software, i.e., selecting sectors to erase.

In the context of PCMCIA storage cards [16] and CompactFlash systems [22, 29], the Flash Translation Layer (FTL) was developed as a means to expose NAND flash through a traditional block device abstraction. The FTL became the central component of SSD firmware [4].

Kwak et al. [26] describe the components of a layered SSD firmware: (a) hardware abstraction modules for the host-interface controller and the storage controllers, (b) an NVMe manager that handles the NVMe commands and forwards I/O commands (i.e., reads and writes) to the (c) Flash Translation Layer; (d) a low-level scheduler that determines the execution order of flash commands and DMA operations requested by the FTL. This bare-metal firmware is single-threaded, it *"fetches a host command, call a command processor, and run the low-level scheduler in the same big loop"* [26].

Early work on multi-threaded FTL [23] was based on a simple multiprocessing model. The design in [23] is based on 5 tasks: a monitor and four tasks, each running an instance of the FTL. Semaphores are used to protect shared resources. The proposed FTL was implemented on a custom board, with a 32 bits single-core Cortex-M3 ARM processor, equipped with the micro-controller operating system μ C/OS-II [27], a preemptive real-time multitasking operating system. On a single core, multi-threading did not improve SSD firmware performance.

Zhang et al. [52] showed that replicating instances of the layered firmware on multiple cores did not scale well. With one instance per core, each handling an NVMe request, the synchronization overhead due to contention on shared meta-data results in poor scaling. To address this problem, they designed DeepFlash, that decomposes the layered firmware into seven operations to which multiple threads are associated. The association of one operation and multiple threads is denoted a *stage*. Queues are used for inter-stage communication. DeepFlash's *staged pipeline* design is representative of modern multi-threaded SSD firmware [21, 23, 47, 52].

DeepFlash [52] takes on the challenge of scaling to 1MIOPS on the Intel MIC 5120D accelerator¹, equipped with 60 lightweight in-order cores, with 4 hardware threads per core, running at 1GHz. The underlying assumption is that there are enough cores to schedule all active stages, thus implementing a form of pipeline parallelism. The performance

¹<https://www.intel.com/content/www/us/en/products/sku/75801/intel-xeon-phi-coprocessor-5120d-8gb-1-053-ghz-60-core/specifications.html>

evaluation shows that 12 cores, i.e., 48 hardware threads, are used to achieve 1MIOPS. The paper claims that DeepFlash *can run with fewer cores if they are more powerful* without further explanations². The DeepFlash paper does not mention how threads are managed on the MIC 5120D. However, the developer guide for this platform[5] reveals that the software stack is based on a flavour of embedded Linux. DeepFlash thus introduces two levels of scheduling: from stages to threads (managed by DeepFlash), and from threads to cores (managed by embedded Linux).

2.2 Programmable SSDs

Twenty years ago, Jim Gray’s expansive vision was to *put Everything on the disk controller* [13]. His argument, inspired by pioneering work on *active disks* [42], was that disk controllers were becoming powerful enough to be programmed with application logic.

In 2019, Do et al. [8] presented a variant of this argument, and predicted the emergence of programmable SSDs³ in future cloud data centers to (a) develop agile and flexible storage interfaces adapted to a given workload, to (b) move compute close to data thus minimizing data movement and reducing host CPU load, and to (c) secure computations in the cloud with SSDs as trusted enclaves.

Examples of new storage interfaces include (a) variations of the memory interface [7], (b) daemons that are launched by a host and executed when some events occur, e.g., piggybacking index creation [25] or sorting [40] on write commands, (c) applications that stream data back to the host [28], possibly on a schedule, or (d) specialized functions, e.g., pointer chasing, simple string search or database scan and filtering [14], graph processing [33], or machine learning [24, 38].

Do et al. argue that *programming SSDs is becoming easier, with the trend away from proprietary architectures and software runtimes and toward commodity operating systems (such as Linux) running on top of general-purpose processors (such as ARM and RISC-V)*. We share this vision. While we agree with the need for a commodity operating system well-suited for SSD programming, we argue in the next Section that embedded Linux is not a good fit.

Willow [44] was a prototype SSD programmable composed of a network of *storage processors*. Each storage processor is a microprocessor with access to non-volatile memory and a programmable DMA engine. The Willow prototype is composed of 8 MIPS softcores, running at 125 MHz on 4 FPGA boards. Pairs of softcores, co-located on an FPGA board, access 8GB of RAM via a custom memory controller,

²Note that Due et al. [10] claim that they experimented with DeepFlash on a quad-core ARM processor, using pthread with an unspecified operating system, without any detail.

³Note that we do not consider computational storage devices [18] as programmable SSDs, as they are programmed against the external NVMe interface of SSDs. We denote programmable SSDs the classes of re-programmable and co-designed SSDs identified by Lerner et al. [30].

configured to emulate non-volatile phase-change memory. Note that this architecture is a departure from the traditional SSD controller architecture from Figure 1. Each softcore runs a lightweight operating system, *SPU-OS*, that *provides simple multi-threading*. SSD applications, denoted SSD Apps, run atop SPU-OS. The conventional FTL is no longer the central component, it is just an SSD App. Most of the paper covers the nature of the interactions between hosts and SSD apps (via RPC), but it only describes the protection features of SPU-OS in details. How SSD apps threads are scheduled by SPU-OS is not discussed. SSDApps provided specialized interfaces [44], an atomic log commit/abort interface and a key/value interface, in addition to the traditional block device interface.

Biscuit [14] incorporated a run-time system with a traditional SSD firmware to support user programs, denoted SSDlets. The firmware is accessible from SSDlets via a file system abstraction. Each SSDlet is a data processing task, with input and output ports, implemented as bounded queues. Cooperative multithreading is used to schedule SSDlets. SSDlets were defined for pointer chasing, simple string search or database scan and filtering. Biscuit ran on an existing SSD controller with two ARM Cortex R7 cores (11-stages, superscalar, out-of-order pipeline with branch prediction) running at 750 MHz with L1 cache. The controller does not support cache coherence, so: (i) SSDlets that belong to a same application are scheduled on the same core, as a result there is no need for locking to protect the shared queues connecting them, and (ii) shared queues, connecting SSDlets and the host or SSDlets from different applications, are not thread-safe.

2.3 Take-aways

We identify two main take-aways: (1) Embedded operating systems are used in the context of existing multi-threaded firmware and programmable SSDs. The interplay between SSD controller features (number of cores, memory hierarchy), operating system and firmware is crucial but not thoroughly explored in the literature.

(2) While the staged pipeline model improves scalability in multi-core SSD firmware, it introduces three fundamental limitations. First, threads within a stage share mutable data structures, requiring careful and brittle synchronization that must be re-engineered with every firmware change. Second, the model embeds platform-specific assumptions—such as cache-coherent memory and OS thread scheduling—directly into the firmware logic, making portability difficult. Third, the interactions between pipeline stages, thread counts, memory hierarchy, and scheduler behavior are opaque.

3 SSD Operating Systems

Why not use Embedded Linux as an SSD operating system? To answer this question, we first consider the design goals

and design constraints for an SSD operating system. We characterize the limitations of existing systems and identify the hypothesis we study with σ .

3.1 Design Goals

Multiprocessing. SSD software should be able to leverage the throughput of the underlying flash chips and to handle requests from the host at PCIe speed. Modern flash chips operate at a few giga transfers per second (GT/s), while PCIe Gen6 operates at 64 GT/s. At this microsecond scale, parallelism is of the essence: pipelining, asynchronous access to (non-volatile) memory, vectorization and task parallelism (possibly with specialized hardware) should be considered. Conversely, overhead and contention are the enemies. Reducing contention was the key motivation for re-engineering the FTL into stages well-suited for many-cores in DeepFlash [54]. *The operating system should support efficient multiprocessing, with minimal overhead.*

Footprint. *The footprint of the embedded operating system should be minimal to minimize resource consumption and cost.*

Evolution. At microsecond-scale, software should be on the control path, i.e., handling command and meta-data, not on the data path. We consider that any processing on the data path should be performed through hardware accelerators, as data is moved in or out of RAM. Note that the kind of meta-data processed in software largely depends on the command sets, e.g., logical-to-physical page table in a FTL, vertex-to-page table in GraphSSD [33] or an embedding cache and embedding operation buffer in RecSSD [50]. *The operating system should efficiently support the integration of new accelerators and the manipulation of complex data structures.*

Testability. The JEDEC JESD 218 standard [17] regulates SSD data integrity (measured as uncorrected bit error rate) and SSD endurance (measured in Terabytes Written or Drive Writes Per Day). SSD vendors provide warranties based on this standard. The underlying assumption is that SSDs are bug free. However, SSD firmware vulnerabilities are not uncommon (e.g., the most recent entry in the CVE database at the time of writing is from March 2025 [45]). In the context of programmable SSDs, revamping the concept of warranty is a business issue. Improving the testability of SSD software is a fundamental technical issue and an open problem.

Recently, deterministic simulation testing has emerged as a means to systematically explore the failure space of distributed systems [9, 55]. The approach pioneered by FoundationDB [55] is to run the actual system with randomized synthetic workloads and fault injection, in a deterministic discrete-event simulation. The failure space can be explored thoroughly with millions of (concurrent) simulation runs, and any bug can be reproduced. The key requirement is that the system is deterministic and that sources of non-determinism in the environment are simulated. Given a random seed, a simulation run (and thus any bug that is found)

can be reproduced. *The operating system should support a deterministic simulation framework.*

I/O Determinism. *Predictable latency on flash storage is a long-pursuit goal [32].* Non-determinism and unpredictability in SSDs are essentially due to contentions within the SSD controller [31]. The NVMe I/O determinism interface enables an SSD to switch between a *Predictable Latency Mode* and a *Busy Mode*. In effect, SSDs that support this feature schedule garbage collection (GC) in busy mode only. There are no worst-case execution time guarantees in predictable latency mode. *Can such guarantees be provided? At what cost in terms of performance?*

3.2 Design Constraints

Memory model. In any multiprocessing system, a memory model defines correctness for shared memory accesses [46]. For instance, RISC-V provides weak memory ordering [48]. If the SSD operating system is to expose a stronger memory model, it should enforce it. To do so, the operating system does (or does not) rely on the availability of hardware atomic primitives and cache coherence.

Predictable timing. There is an obvious tension between performance and timing predictability. Modern processors and memory subsystems include features such as out of order execution, branch prediction, frequency scaling or multiple levels of instruction- and data-caches that are obstacles to predictability [49]. These are optimized for maximum average-case throughput. Alternatively, predictable architectures (e.g., PRET [36]) are designed for a predictable worst-case execution time. They have the following properties: (a) no timing anomalies, (b) software-managed scratchpad memories, and (c) deterministic arbitration on the memory bus.

3.3 Existing systems

Table 1 summarizes the characteristics of Embedded Linux and $\mu\text{C}/\text{OS-II}$ mentioned in Section 2, as well as FreeRTOS (used in [35]), based on the goals and constraints identified above.

$\mu\text{C}/\text{OS-II}$ is designed for single-core, so it is not relevant for our purpose.

Embedded Linux is usually best-effort, but it can be run in deterministic mode with PREEMPT_RT⁴. It exposes a well-known programming model based on POSIX-threads. Its footprint is in the order of MBs and the overhead associated to transition across threads is measured in 10s of μsec . It also requires a MMU for virtual memory, as well as atomics and cache coherence to synchronize accesses to shared kernel data structures. We conclude that Embedded Linux is not a good match for SSDs.

FreeRTOS is much more lightweight and efficient. Its programming model is based on tasks. It has low footprint and

⁴<https://realtime-linux.org>

Property	Embedded Linux [3]	FreeRTOS [12]	μ C/OS-II [27]
Multiprocessing	Thread-based; symmetric multiprocessing across many cores	Task-based; symmetric multiprocessing across few cores	Single-core only
Footprint	≥ 8 MB RAM, ≥ 32 MB flash	~ 2 – 5 kB RAM, ~ 5 – 15 kB flash	~ 4 kB RAM, ~ 6 – 24 kB flash
Evolution	Device driver model; device tree	No driver model	No driver model
Testability	No deterministic scheduler	deterministic preemptive scheduler; portable layer for host simulation	deterministic scheduler
Memory Model	Virtual memory with MMU	Flat physical address space	Flat physical address space
Need for Atomics	atomics required	atomics required	mutual-exclusion through global interrupt disable
Predictable Timing	>10 μ sec across thread executions or between interrupt and thread execution with PREEMPT_RT	millisecond-level tick resolution; sub- μ sec interrupt latency	worst-case execution time (WCET) analysis; μ sec latency

Table 1. Assessment of Embedded Linux (with PREEMPT_RT), FreeRTOS and μ C/OS-II.

low overhead. It was originally designed for single core, but has been extended for symmetric multiprocessing across a few cores with a global spinlock guarding the list of ready tasks. This design does not scale to many cores. It is not a good match for SSDs.

3.4 Design Space

Based on the constraints and goals above, we map the design space, along two dimensions:

- **SSD regime:** We distinguish two regimes – *best-effort* that favours performance without guarantees, and *deterministic* that provides determinism and may sacrifice some performance.
- **Hardware platform:** We distinguish two classes of hardware platforms – *coherent* optimized for maximum average-case throughput that provides atomics and coherent memory, and *predictable* designed for worst-case execution analysis and predictable timing without support for atomics or coherent memory.

We design σ to explore these quadrants. More specifically, we derive the following research questions from our central hypothesis, expressed in Section 1:

1. Can σ deliver sub- μ sec overhead?
2. Can σ support a FTL running at microsecond-scale?
3. What is the performance penalty for a deterministic execution?
4. Can meaningful worst case execution time guarantees be defined for a FTL?

Before we tackle these questions in Section 7, let us detail the design of σ .

4 σ

We propose σ as an embedded operating system for Solid-State Drives, i.e., a narrow waist supporting various SSD applications (including the traditional FTL) atop a range of RISC-V-based SSD controllers.

4.1 System Model

We consider the slice of the SSD controller represented with a white background in Figure 1. Our system definition thus considers accelerators, host-interface and storage controllers as part of the environment. Put differently, the management of data movement through DMA engines is entirely based outside our system.

This assumption is consistent with previous work on (a) Host-interface Controllers (e.g., OpenExpress [20] or AMD NVMe Target Controller⁵) that encapsulate host DMA, PCIe transaction management and NVMe protocol management, and (b) Storage Controllers (e.g., Babol [35]) that encapsulate flash DMA, flash operation management and low-level scheduling. Host-interface and Storage controllers are abstracted through drivers that may combine hardware and software components.

This system model generalizes to any RISC-V-based system expected to manage data movement.

4.2 Abstractions

σ supports a range of platforms that abstract the underlying SSD controller. For each platform, a collection of drivers is defined for accelerators and peripherals. Each platform defines a compilation target with a well-defined RISC-V ISA and ABI.

⁵<https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/ef-di-nvmetc.html>

To support multiprocessing at microsecond-scale, σ provides two central abstractions:

Memory regions. σ supports region-based memory management. Memory regions are continuous blocks of physical memory based on sections of the memory map (e.g., RAM or scratchpad). They are defined statically, at compile time. At run-time, programs manage memory through an application-specific allocator, e.g., that advances an offset through the block (a constant-time operation) and reclaims memory by resetting the offset to zero. Memory regions may be accessed by a single program or it may be shared by several programs. σ defines two kinds of memory accesses: *sequentially consistent* regions, over which σ enforces a single, global order over all memory accesses, and *relaxed* regions where no consistency model is enforced. With region-based memory management, we *allocate all memory at startup*⁶ for performance and predictability.

Tagged-token dataflow graph. An σ application is a tagged-token dataflow graph, where each vertex is a *task* and each edge is a directed channel that carries a *token*. Each token contains a *tag* associating a function pointer (corresponding to the task to be executed) and a context pointer (the task input).

We distinguish different types of tasks: (i) *sources* that do not accept incoming tagged-tokens and (ii) *stages* that manipulate a single tagged-token at a time. We consider three variants of a stage. A stage with (a) zero outgoing edge is a *sink* that only consumes tagged-tokens, a stage with (b) multiple outgoing edges is a *conditional* that transfers a token on one of the outgoing edges based on the incoming tagged-token (and possibly its state), and a stage with (c) multiple incoming edges is a *merger* that consumes one of several possible tagged-tokens.

An σ application is a forest of multi-source, multi-sink direct acyclic graphs with a bounded number of in-flight tokens. Consider a SSD, where the host-interface controller divides incoming NVMe commands into multiple sliced command requesting fixed-size data [26], separate subgraphs can be dedicated to each slice. We consider pairs of source and sink associated to any external component that is accessed asynchronously. The sink submits an asynchronous command and the source polls for notification. New tokens are generated sporadically by a source and consumed by a sink. We discuss the bounds on the number of tokens in Section 5.2.

Each task is placed on a core and executes to completion. For instance, a stage dequeues a token, performs the computation based on the tag it contains, and enqueues a new token for the next task in the dataflow graph.

At any time, a single application executes on a device. Conceptually, an application can be extended with new tasks and new edges, or it can be modified via edge rewiring. For

now, we consider that such modifications result in a new application. Run-time modifications of the dataflow graph are a topic for future work.

We adopt the idea of tagged-token data flow, pioneered at MIT in the 80s [1], as a generalization of the staged pipeline design from DeepFlash [52]. Instead of a fixed number of operations, each associated to a given number of threads, and arranged in pre-defined pipelines, σ supports a user-defined graph of programmed tasks that it directly maps onto the available cores. This is different from the thread model of Willow, where communication across threads is managed by the programmer, and of the stream processing model of BISCUIT, where data flows across SSDlets, as opposed to tagged-tokens in σ .

4.3 Programming Model

```

1 typedef void *tag_context;
2 typedef void *(tag_function) (void *)
3
4 typedef struct {
5     tag_function *function;
6     tag_context context;
7 } tag;
8
9 typedef struct {
10     int16_t edge_idx;
11     tag_context context;
12 } task_output;
13
14 typedef void ( init_fn ) ( void ) ;
15 typedef task_output ( task_fn ) (tag_context);
16 typedef tag_context ( poll_fn ) ( void ) ;
17
18 struct task {
19     uint8_t type; //e.g., source, stage
20     uint16_t id;
21     init_fn *init_fn;
22     poll_fn *poll_fn; // null for stages
23     task_fn *task_fn;
24     uint8_t num_out_edges;
25     struct task *out_edges;
26 };

```

Listing 1. σ task representation

A program describes a tagged-token dataflow graph. To this end, σ defines data structures and function interfaces, as described in Listing 1⁷.

Each vertex in the dataflow graph is an instance of a `struct task`, that contains an array of outgoing edges to other task instances. Each task has a given type (as described above), a unique id and an initialization function that is called at deployment time. Sources have a non-null pointer to a `poll_fn` (that takes no argument and returns a tag context) to test whether a resource is available from the environment (e.g., a flag or a doorbell). The computation performed by a task is represented by the `task_fn` function that takes a tag context as input and returns as output a pair associating context and outgoing edge index (for conditionals a single token is issued on one of the possible outgoing edges).

⁷The listing is a slight simplification of the σ code, which is kept private to comply with double blind reviewing.

⁶<https://tigerstyle.dev>

4.4 Scheduling

As stated above, each task runs to completion. We put the burden of keeping tasks short on the programmer (or to the compiler of a firmware development kit where a programmer relies on a domain-specific language – this is a topic for future work). We thus trade flexibility for simplicity and performance. In fact, we argue in Section 6.3 that the discipline required for task design is a sound guiding principle for SSD application designers.

According to Davis et al. [6], multiprocessor scheduling addresses two problems: (i) task placement onto a core, and (ii) task activation. The rest of this section focuses on task placement. We cover task activation task activation as part of the execution model in Section 5.1.

We consider a *partitioned scheduling algorithm* [6], where each task is statically allocated to a core. We thus reduce the complexity of task transition (as the core placement of the next task can be embedded in a tag, rather than computed by a dispatcher thread). The cost stems from potential skewed task placement: either too few task causing idle cores or too many tasks causing bottlenecks.

The static placement of tasks onto cores make it possible for us to deal with concurrency across tasks accessing shared memory regions: there is no need to synchronize tasks that are guaranteed to execute serially on a core.

We annotate tasks in the dataflow graph with the shared memory regions that they access. From these annotations, we derive a conflict graph. Dataflow graph, conflict graphs and the number of cores are the input to our task placement algorithm, described in Algorithm 1. The algorithm guarantees that each source is placed on a different core and that all tasks that conflict are placed on the same core. The rest of the tasks are placed to balance the load. As we consider multiple tokens, whose generation is sporadic, we cannot rely on the precedence relationship from the dataflow graph to optimize placement (at any point in time, any two tasks in the dataflow graph might be ready to be executed).

If there are enough cores, in best-effort mode, we consider an alternative *one-to-one* mapping, where each task is mapped onto a dedicated core.

4.5 Library

σ is written in C and packaged as a library. An application is a collection of relocatable object files, that complies with the programming model, and compiled any language that is ABI compatible with C. σ supports statically linked applications, or applications that are dynamically linked at run-time. In the latter case, the runtime linker processes relocation entries. Both σ and the application run in machine mode. Loading code that modifies a statically linked application, and verifying the code that is loaded are topics for future work. At boot time, execution begins with σ initialization

Algorithm 1 Conflict-aware task-to-core mapping

Require:

$G = (T, E_d)$: dataflow graph
 $C = (T, E_c)$: conflict graph
 m : number of cores

Ensure:

MAP : $T \rightarrow \{0, \dots, m-1\}$

```

1: Let Sources  $\leftarrow \{t \in T \mid \deg^-(t) = 0 \text{ in } G\}$ 
2:  $\mathcal{K} \leftarrow \text{CONNECTEDCOMPONENTS}(C)$ .
3: // Assign sources to dedicated cores
4: Assign each  $s \in \textit{Sources}$  to a unique core
5: // Assign conflict components to cores
6: for each  $K_i \in \mathcal{K}$  do
7:   if  $K_i \cap \textit{Sources} \neq \emptyset$  then
8:     Assign all tasks in  $K_i$  to the core already assigned
9:   else
10:    Assign all tasks in  $K_i$  to the least-loaded core.
11:    Update the assigned core's load accordingly.
12: // Assign any remaining unassigned tasks
13: for each unassigned  $t \in T$  do
14:   assign  $t$  to the least-loaded core and update its load
15: return MAP

```

(i.e., registers setup, memory initialization, drivers initialization) followed by application initialization.

5 Execution Models

We design three mechanisms for σ ' execution model, i.e., how tokens are exchanged and tasks activated: (1) In *best-effort mode*, we assume an underlying coherent platform, supporting atomics to provide traditional synchronization for efficient token exchange, without guarantees. (2) In *deterministic mode*, we do not make assumptions about the underlying platform. Our goal is to define an execution model that guarantees a deterministic history of task activations. To this end, we introduce a tick system, designed to improve testability. (3) In *synchronous mode*, we extend the deterministic tick system so that all ticks have the same fixed-length duration. The goal of such a synchronous system is to support worst case execution time analysis on a predictable platform.

5.1 Best-Effort Mode

The best-effort σ execution model relies on cache-aligned, mutiple-producers/single-consumer lockless rings for the exchange of tagged-tokens and task activation. We prefer lockless rings to bounded buffers (e.g., as used in DeepFlash [52]) to minimize overhead under the assumption that core utilization is high.

Lockless Rings. There is one lockless ring per core. It contains tagged-tokens used to activate the tasks mapped onto that core. The consumer is a local task dispatcher that

dequeues a token from the ring, and calls the associated task based on the tag it contains. After task completion, the dispatcher enqueues a token produced with the task output. The token is enqueued on the local lockless ring or on another core depending on the placement of the next task in the dataflow graph. Task dispatchers from multiple cores may write tagged-tokens on a lockless ring concurrently.

When a source is placed on a given core, the associated dispatcher polls its environment in an application-specific way (using `poll_fn` – see Listing 1). If the poll function succeeds, the source is activated (i.e., the task function associated to the only source mapped to the core is called with the tag context returned by the poll function). To guarantee liveness, the dispatcher stops polling after a given number of source activation is reached.

The lockless rings implementation, inspired from DPDK⁸ relies on pairs of pointers (head and tail) for producer and consumer. Coherent memory and atomics are required to synchronize access to the producer head (through compare-and-swap) and to make new pointer values visible to all cores (through memory fences). The order in which tasks are activated is non-deterministic because it depends on the order in which producers acquire the head pointer with a compare-and-swap.

5.2 Deterministic Mode

For deterministic simulation, it should be possible to reproduce the state of the system at any given logical time of an execution. We define σ *state determinism* as the combination of (i) *deterministic history*: the sequence of task executing on each core only depends on the inputs (i.e., the tokens generated by sources), (ii) *deterministic shared memory*: the order of read/write operations to shared memory and their outcome is determined by the inputs, and (iii) *deterministic channels*: the tokens exchanged on channels across tasks only depend on the inputs.

Multicore Tick System. To guarantee state determinism, we design a tick system. A tick is defined as the time it takes to execute one active task per core. Thus the tasks executing on different cores within the same tick are logically simultaneous. Ticks may have different length (depending on the slowest task to complete).

At each tick, on each core, the task dispatcher executes the following steps:

Step 1. At the beginning of each tick, the task dispatcher drains the tagged-tokens enqueued during the previous tick into a local queue (see Figure 2). This is always possible because the size of this local queue determines the credit-based flow control described below. It is thus guaranteed that the number of in-flight tokens is always lower than the size of the local queue. The tag-tokens enqueued in the previous ticks are stored in a tag-buffer, i.e., an array of

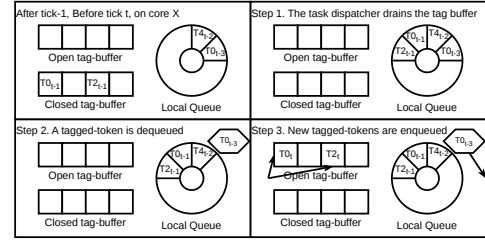


Figure 2. State of tag buffers and local queue during the three steps executed by a task dispatcher during a tick.

tagged-token slots of size m , where m is the number of cores in the system. The tag-buffer is drained in core order. That order is respected in the local queue. We consider two tag-buffers per core: one is closed and contains tagged-tokens written at tick $t-1$, the other is open and can be updated at tick t .

Step 2. The task dispatcher then dequeues a tagged-token from the local queue and executes the associated task. The order of task executions on a core is thus determined by the order of tagged-tokens in the local queue.

Step 3. At the end of each tick, the task dispatcher enqueues the tagged-token to the next task in the dataflow graph, i.e., it writes the tagged-token on the tag-buffer of the core on which the task is placed. It is always possible. The structure of the tag-buffer and the credit-based flow control guarantee that there are never any conflicts across cores when writing to a tag-buffer. Finally, a memory barrier enforces that the effects of all tasks from the current tick are visible to all tasks from the next tick. This barrier does not rely on atomics. It is implemented through a barrier based on a centralized array of flags. Each core writes the current tick id at a given index when a task finishes executing, and when transitioning to the next tick, each core reads all flags and spins if a flag is not set to the current tick [34].

Credit-Based Control Flow. The size of the local queue is fixed. This size is the upper bound on the number of in-flight tokens in the dataflow graph. To enforce this guarantee, we implement a form of credit-based flow control, that maintains the following invariant: *number of in-flight tokens + number of credits = local queue size*. The number of credit is made visible to all cores after the tick barrier. It is updated after each tick: A sink that consumes a tagged-token increases the number of credit by 1. A source only generates a tagged-token if the number of available credits is strictly larger to the number of sources in the system. This guarantees that either all sources can generate a new token (and decrease the number by 1) or none can (without synchronization). This is necessary to avoid source starvation (liveness). Note that this flow control mechanism prevents (a) buffer overflow (safety) and (b) circular wait deadlocks (liveness).

σ state determinism. The combination of the synchronous tick system, credit-based flow control (see Section 5.1)

⁸https://doc.dpdk.org/guides/prog_guide/ring_lib.html

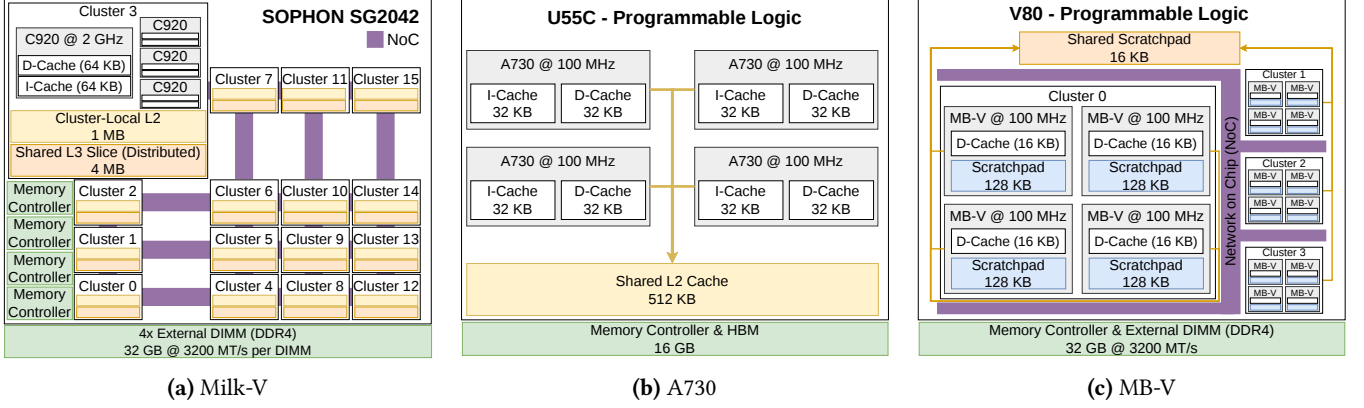


Figure 3. Experimental platforms.

and conflict-aware task placement (see Algorithm 1) guarantee state determinism. First, the tag-buffers on all cores at tick t store all in-flight tokens generated at this tick. All inflight-tokens generated at previous ticks are stored on the local queues. This ensures channel state determinism. Second, the task dispatcher always drain the tag-buffer in core order. This order is preserved in the local queue. As a result, the schedule of tasks is deterministic on each core. Third, the conflict-aware placement algorithm guarantees that there are no conflicting concurrent accesses to shared memory. The memory barrier at the end of each tick enforces a global order across ticks. This guarantees deterministic shared memory.

5.3 Synchronous Mode

We propose a synchronous execution model to support the definition of worst-case execution time guarantees [51]. Our hypothesis is that a synchronous system operating with fixed tick duration can support the definition of such guarantees.

The tick system introduced in the previous Section is made synchronous through timer interrupts enforcing fixed-length ticks. If a task dispatcher is done executing a task, it waits for the timer interrupt. The interrupt handler checks whether the task finished executing (and emitted a new token) or not. In the former case, a new task is dequeued in the next tick. In the latter case, task execution simply resumes after the tick, but our system is no longer synchronous.

How can we define an appropriate tick duration? It should be defined to handle transfers from the host-interface controller and from the storage controller. For instance, assuming a saturated channel composed of 16 flash packages operating at 1.2 GT/sec, the interarrival time of 4 KB transfers is 3.4 μ sec (and the number of in-flight tokens for our credit-based flow control should be 16). If we consider a channel that is 85% saturated, we get 4 μ sec between transfers. Also, the NVMe Target Controller has an effective bandwidth of 8 GB/sec, i.e., the interarrival time between two 4 KB transfers is about 0.5 μ sec. Assuming requests are handled in batches,

e.g., dispatched in parallel over 8 channels, that gives us also 4 μ sec in between batches.

The synchrony hypothesis states that tick duration should be lower or equal to the interarrival time of events from the environment [51]: 4 μ sec, assuming a balanced Gen4x4 SSD with 8 channels, each connecting 16 flash packages.

As transfer times for flash and PCIe keep on doubling, increasing batch size, i.e., the number of external events handled per tick (as we have already assumed for NVMe commands), is the only way to keep tick duration stable. Alternatively, tick duration can be increased to accommodate larger batches.

6 Experimental Framework

6.1 Platforms

We experiment with three different platforms, all based on RISC-V 64 bits processors, illustrated in Figure 3:

Milk-V: based on the SOPHON SG2042 SoC, equipped with 64xT-Head XuanTie C920 cores and a coherent memory subsystem. The 64 cores are arranged in 16 clusters of 4 cores each containing a dedicated L2 cache and a slice of a shared L3 cache, over 4 DIMMs of DDR RAM. The C920 core is dual-issue, 12 stages pipeline with branch prediction and out-of-order execution, running at 2 GHz.

A730: an AMD/Xilinx U55C FPGA, equipped with 4xCo-dasip A730 softcores and a coherent memory hierarchy with two levels of cache over HBM. The A730 softcore is dual-issue, 9 stages pipeline with branch prediction and fully-in-order execution, running at 100 MHz. We use this platform to explore how much can be achieved with softcores.

MB-V: an AMD/Xilinx V80 FPGA, equipped with 16x AMD/Xilinx MicroBlaze-V softcores, equipped with non-coherent memory: 128kb core local scratchpad, 16 KB scratchpad shared by all cores and shared DDR4 RAM. The MB-V softcore is a single-issue, 5 stages pipeline without branch prediction, with fully-in-order execution, running at 100 MHz. This is an instance of predictable architecture.

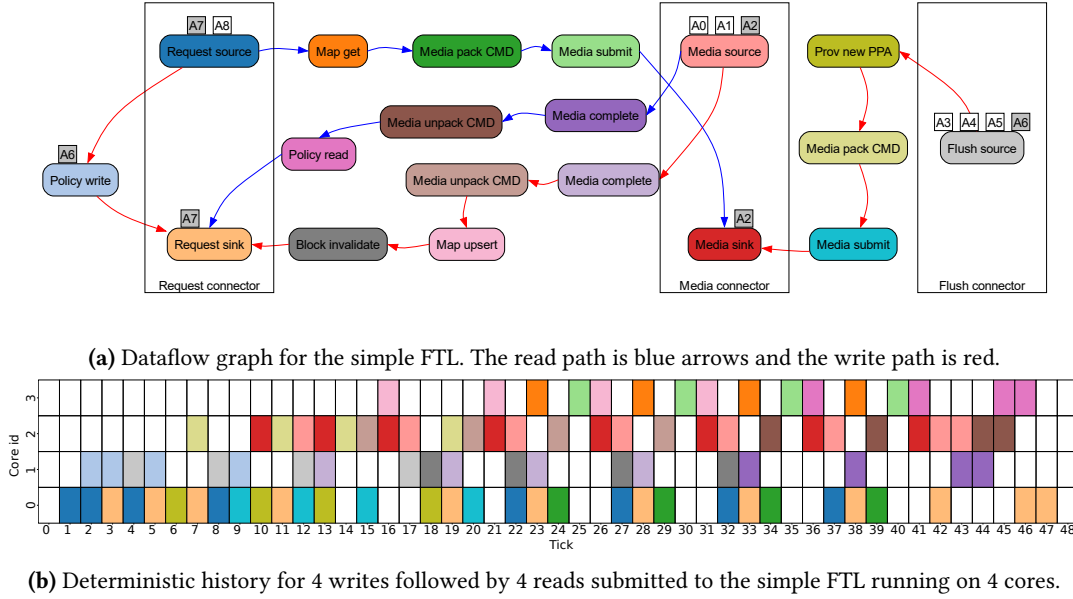


Figure 4. Simple FTL.

6.2 σ Implementation

σ is implemented in 16K SLOC of C (and 800 SLOC of assembly). At deployment time, instructions and core-local data structures fit in 128 KB, while shared data structures fit into 16KB. In addition to the three platforms above, σ runs on qemu with both 32-bit and 64-bit versions of RISC-V.

For each platform, σ provides (1) *driver implementations* (e.g., for the UART). Defining drivers for host-interface controller and storage controller is a topic for future work; (2) *assembly start-up and boot script*. At startup, σ loads the binary into the appropriate memory regions, initializes hardware registers and launches the task dispatcher on each core (supporting run-time modification of σ dataflow graph will require a dynamic loader that takes an object file and performs loading and relocation – this is ongoing work); (3) *constants*: e.g., number of cores, memory regions, ABI and ISA string.

6.3 FTL

We developed two versions of a simple FTL, one in C that we use for the experiments (5.5K SLOC) and one in Rust (1.3K SLOC)⁹. These FTL variants run on the three platforms: σ is a narrow waist.

The initial dataflow graph for the simple FTL, shown in Figure 4, is based on 19 tasks, one task per function (we refine this design in Section 7.2). There are three sources/sink pairs, that we denote *connectors*: one for interacting with the host-interface controller (Request), one for interacting with the storage controller (Media), and one for managing flushes from the write cache (Flush). The memory regions accessed

by each task are illustrated by boxes denoted A_i , in white if it is local and grey if it is shared.

The logical-to-physical (L2P) mapping table is implemented as an array-indexed map placed in RAM. Both lookup and insertion are constant time. The L2P maps logical block addresses (lba) to physical block addresses (ppa). On every update, a new physical page address is allocated for the data. This allocation is handled by the provisioning task (Prov new PPA). Each NVMe request, containing an lba and the ppa it is mapped to, is represented as a task context.

There is no garbage collection in this simple FTL. Also, there is no data movement as it is managed outside our system. We focus exclusively on meta-data management. As a result, the performance numbers from the next Section do not represent actual performance numbers, they just indicate the point at which σ becomes a bottleneck.

Figure 4 shows the history obtained in deterministic mode for the simple FTL executing 4 writes followed by 4 reads on 4 cores. We can simulate millions of diverse workloads on the simple FTL running on σ and replay the same history given the simulator seed. In deterministic mode, σ is thus the foundation of a deterministic simulation framework.

7 Experimental Results

Our experiments address two sets of questions: (1) Does σ operate at microsecond-scale? (2) How fast can σ run a FTL in best-effort mode? What are the trade-offs involved in deterministic or synchronous mode?

7.1 σ Overhead

To answer the first question, we quantify the overhead of running σ with a simple dataflow graph, using empty task

⁹The source code will be made publically available.

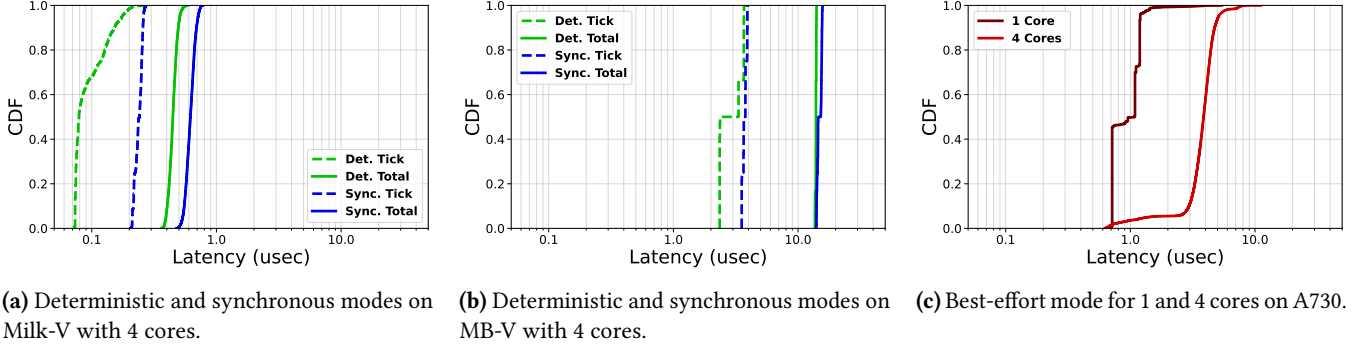


Figure 5. Distribution of tick and task transition overheads. Note the logarithmic x-axis scales.

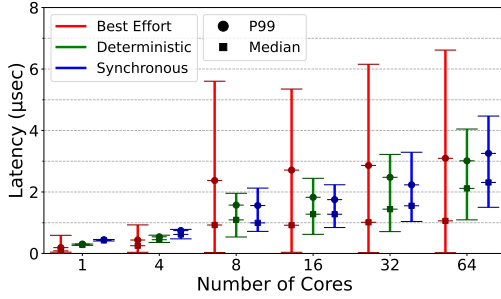


Figure 6. Task transition overhead for varying number of cores in best-effort, deterministic and synchronous modes on Milk-V.

functions. This dataflow graph is composed of a source and a sink, connected by empty stages. We keep the ratio between the overall number of tasks and the number of cores constant (this ratio is 1:1, except on 1 core where we consider 4 tasks). We collect 30K samples with each platform and vary the number of cores from 1 (the number of sources) to the maximum number of cores. We measure *task transition overhead*, i.e., the time elapsed between the return of a task function and the start of the next task function.

Figure 6 shows the task transition latency in best-effort, deterministic and synchronous modes on the Milk-V as the number of core is varied from 1 to 64. The maximum latency is sub- μ sec for all three designs on 1 and 4 cores, i.e., within a cluster. The overhead latency increases sharply for the three designs when the number of cores increases above cluster size. This is because memory accesses to lockless rings or tag buffers target the L2 cache within a cluster, while they target the L3 cache across clusters.

As expected, the distribution of overhead latencies increases significantly in best-effort mode, when the core count is larger than 4. The distribution remains tighter in the two other modes. On the other hand, the median overhead for the best-effort design is stable at around 1 μ sec as the core count increases, which is not the case for the other two designs.

Figure 5 shows the distribution of the task transition latencies across modes and platforms. In particular, Figure 5a

shows that the distribution of the task transition latencies with 4 cores on the Milk-V is concentrated around 0.4 μ sec in deterministic mode, and around 0.6 μ sec in synchronous mode. There is no long tail. This overhead is an order of magnitude smaller than the typical overhead for context switches on Embedded Linux or FreeRTOS (see Section 3).

The 20x factor in processor frequency between Milk-V and both softcores (MB-V and A730) is reflected in an order of magnitude difference in overhead latency. As importantly, the characteristics of the memory subsystem have a very important impact on performance on all platforms.

Overall, the answer to our question is yes on the Milk-V. The overhead of task transition can be kept below 1 μ sec, if we keep the dataflow graph running on a cluster of 4 cores. Across clusters, overhead becomes prohibitive. It is thus more efficient to operate with one dataflow graph partition per cluster, rather than with a large dataflow graph running across clusters on the Milk-V. On softcores, the task transition overhead is too large for operations at microsecond scale.

7.2 FTL Performance

To answer the second set of questions, we run experiments with the simple FTL. First, we focus on the latency of individual task functions in the dataflow graph.

Figure 8 shows the distribution of the latency per task function from the simple FTL running a mix of 30K random and sequential write and read requests on the Milk-V. For all but one function, the maximum latency is well below 2 μ sec. The function that provisions physical addresses (Prov new ppa) has a P99 of 0.3 μ sec and a max of 3.94 μ sec. This is due to a code path with multiple memory accesses that is rarely taken. The implication is twofolds. First, the simple FTL cannot strictly be run within 4 μ sec ticks in synchronous mode. Second, the time it takes to execute a single function is comparable to the task transition overhead. To improve the ratio between useful work and overhead, we collapse each pipeline between connectors into a single stage.

We now turn to FTL performance and focus on 30K requests of well-defined I/O patterns. We measure latency and throughput in the three modes on the Milk-V with 4 cores.

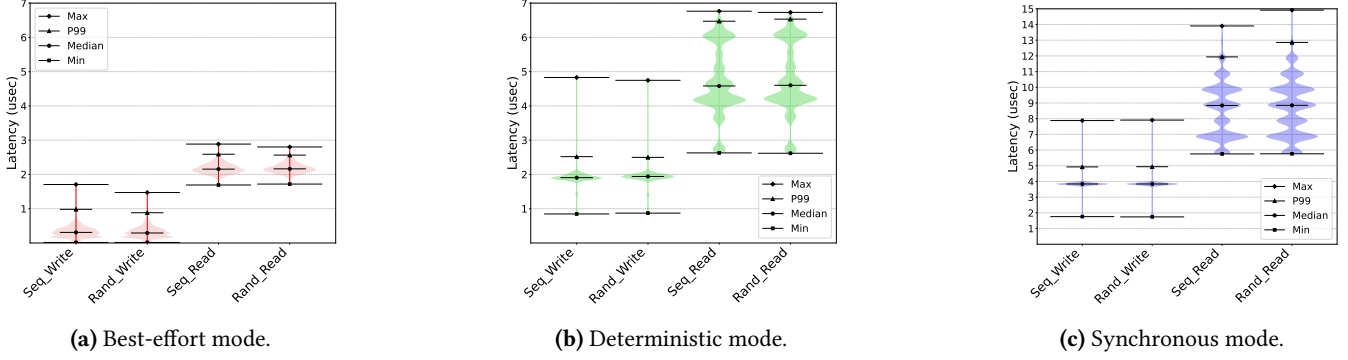


Figure 7. FTL latency with 4 cores on Milk-V. Note that the y-axis scale is different in (c).

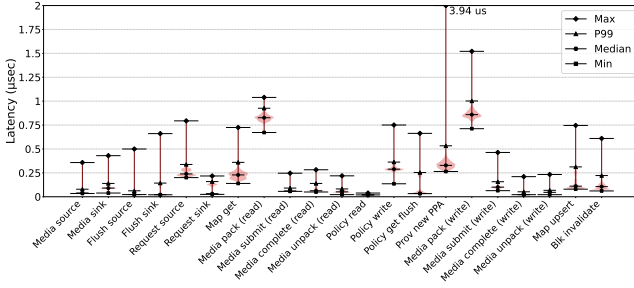


Figure 8. Execution time for task functions on Milk-V.

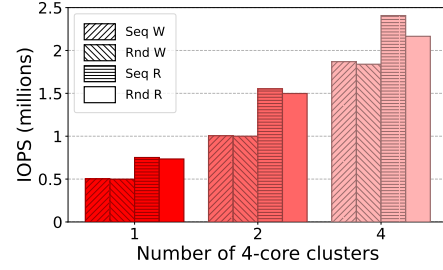


Figure 9. Best-effort IOPS for the simple FTL on Milk-V running on 1x4 cores, 2x4 cores and 4x4 cores.

Figure 7a shows the distribution of FTL latency in best-effort mode, and Figure 7b in deterministic mode. In best effort mode, read and write requests take less than $3 \mu\text{sec}$. It means that the simple FTL, running atop σ would not be a bottleneck in the balanced SSD from Section 5.3.

For now, FTL latency in deterministic mode is about a factor of 2 slower compared best-effort, while throughput (not shown because of lack of space) is 40% lower. Refactoring of the FTL and further optimizations of the task dispatcher should reduce this factor. This is a topic for ongoing work.

It might have been obvious to a careful reader, but the synchronous mode does not guarantee low variance. This is clearly illustrated in Figure 4b, where we see that the first write request completes in 3 ticks, while the second completes in four ticks due to a flush scheduled on core 1 at tick 4. We run an experiment to determine the extent of this variance with a tick duration of $1 \mu\text{sec}$ (well below a duration that guarantees that ticks are never missed). Figure 7c shows that in synchronous mode latency distribution concentrates around values corresponding to number of ticks. We could compute a worst case execution time, assuming a longer tick duration, but such a *predictability* guarantee is meaningless, when variance is high and median latency is at least a factor of 2 higher than in deterministic mode.

Figure 9 shows the FTL throughput in best-effort mode when replicating the FTL on clusters of 4 cores on the Milk-V. With 4x4 cores, σ achieves ~ 2 MIOPS. This experiment

shows the potential of replicating (portions of) the dataflow graph, a variation on the 1:n mapping of tasks to cores from DeepFlash [52]. Much more work is needed to define flexible and scalable replication support as well as support for request batching in σ .

8 Conclusion

We designed σ an operating system for Solid-State Drives. We showed that its programming model, based on memory regions and tagged-token dataflow graph, is well-suited for the most conventional form of SSD application, i.e., the Flash Translation Layer. We ran variants of the FTL written in C and in Rust on three different RISC-V-based platforms. Our experiments showed sub- μsec task transition overhead, an order of magnitude improvement with respect to existing systems, and scalability to millions of IOPS by replicating portions of an application across clusters of cores.

This paper lays the foundation for rich areas of further research. First, we will explore the integration of σ with host-interface and storage controllers to compose a programmable SSD device. Second, we will improve parallelism support in σ including subgraph replication and request batching. Third, the community can now explore the design of data systems-specific SSD applications, and the integration of programmable hardware accelerators and SSD software with σ .

References

- [1] Arvind and David E Culler. 1983. *Tagged Token Dataflow Architecture*. Computation Structures Group Memo 229. Laboratory for Computer Science of the Massachusetts Institute of Technology. <https://csg.csail.mit.edu/pubs/memos/Memo-229/Memo-229.pdf>
- [2] Adrian M Caulfield, Arup De, Joel Coburn, Todor I Mollow, Rajesh K Gupta, and Steven Swanson. 2010. Moneta: A high-performance storage array architecture for next-generation, non-volatile memories. In *43rd Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE. doi:10.1109/micro.2010.33
- [3] Felipe Cerqueira and Björn Brandenburg. 2013. A comparison of scheduling latency in linux, preempt-rt, and litmusRT. In *Proceedings of the Operating Systems Platforms for Embedded Real-Time applications*. https://pure.mpg.de/rest/items/item_2173547/component/file_2173546/content
- [4] Tae-Sun Chung, Dong-Joo Park, Sangwon Park, Dong-Ho Lee, Sang-Won Lee, and Ha-Joo Song. 2006. System software for flash memory: A survey. In *Embedded and Ubiquitous Computing*. Springer Berlin Heidelberg, Berlin, Heidelberg. doi:10.1007/11802167_41
- [5] Intel Corporation. 2013. *Intel Xeon phi coprocessor developer's quick start guide*. Technical Report. <https://www.intel.com/content/dam/develop/external/us/en/documents/intel-xeon-phi-coprocessor-quick-start-developers-guide.pdf>
- [6] Robert I Davis and Alan Burns. 2011. A survey of hard real-time scheduling for multiprocessor systems. *ACM Comput. Surv.* 43, 4 (Oct. 2011). doi:10.1145/1978802.1978814
- [7] Jaeyoung Do, Ivan Luiz Picoli, David B Lomet, and Philippe Bonnet. 2021. Better Database Cost/Performance via Batched I/O on Programmable SSD. *VLDB J.* 30, 3 (May 2021). doi:10.1007/s00778-020-00648-z
- [8] Jaeyoung Do, Sudipta Sengupta, and Steven Swanson. 2019. Programmable solid-state storage in future cloud datacenters. *Commun. ACM* 62, 6 (May 2019), 54–62. doi:10.1145/3286588
- [9] Dominik Tornow and Joran Dirk Greef. 2025. How to write your own Deterministic Simulator. <https://www.youtube.com/watch?v=JoYjiiDZCE>
- [10] Zelin Du, Shaoqi Li, Zixuan Huang, Jin Xue, Kecheng Huang, Tianyu Wang, and Zili Shao. 2024. PipeSSD: A Lock-free Pipelined SSD Firmware Design for Multi-core Architecture. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. ACM, New York, NY, USA. doi:10.1145/3649329.3657384
- [11] FADU Inc. 2026. FADU – Flash Storage Solutions. <https://www.fadu.io>. Accessed: 2026-03-06.
- [12] FreeRTOS. 2026. FAQ - Memory Usage, Boot Times & Context Switch Times. <https://www.freertos.org/Why-FreeRTOS/FAQs/Memory-usage-boot-times-context> Accessed 2026-03-30.
- [13] Jim Gray. 1998. Put Everything on (Future) Disk Controller. It is not if but when. https://jimgray.azurewebsites.net/talks/Gray_NASD_Talk.ppt
- [14] Boncheol Gu, Andre S Yoon, Duck-Ho Bae, Insoon Jo, Jinyoung Lee, Jonghyun Yoon, Jeong-Uk Kang, Moonsang Kwon, Chanho Yoon, Sangyeun Cho, Jaeheon Jeong, and Duckhyun Chang. 2016. Biscuit: A Framework for Near-Data Processing of Big Data Workloads. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. IEEE, Seoul, South Korea. doi:10.1109/ISCA.2016.23
- [15] Eliyahou Harari, Robert D Norman, and Sanjay Mehrotra. 2006. Flash EEPROM system. <https://patentimages.storage.googleapis.com/90/ab/c5/dc2bf218fbdcdb/EP1031992B1.pdf>
- [16] Intel Corporation. 1998. *Understanding the Flash Translation Layer (FTL) Specification*. Technical Report. http://dooros.org/download_documents/FTL_INTEL.pdf
- [17] JEDEC. 2025. JESD 218 - Solid-State Drive (SSD) Requirements and Endurance Test Method. <https://standards.globalspec.com/std/10027789/jedec-jesd-218>. Accessed: 2026-03-05.
- [18] Insoon Jo, Duck-Ho Bae, Andre S Yoon, Jeong-Uk Kang, Sangyeun Cho, Daniel D G Lee, and Jaeheon Jeong. 2016. YourSQL: a high-performance database system leveraging in-storage computing. *Proceedings VLDB Endowment* 9, 12 (Aug. 2016). doi:10.14778/2994509.2994512
- [19] Sang-Woo Jun, Ming Liu, Sungjin Lee, Jamey Hicks, John Ankcorn, Myron King, Shuotao Xu, and Arvind. 2015. BlueDBM: an appliance for big data analytics. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA '15)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/2749469.2750412
- [20] Myoungsoo Jung. 2020. OpenExpress: fully hardware automated open research framework for future fast NVM devices. In *Proceedings of the USENIX Conference on Usenix Annual Technical Conference (USENIX ATC'20)*. USENIX Association, USA. <https://www.usenix.org/conference/atc20/presentation/jung>
- [21] Myoungsoo Jung, Jie Zhang, Ahmed Abulila, Miryeong Kwon, Narges Shahidi, John Shalf, Nam Sung Kim, and Mahmut Kandemir. 2018. SimpleSSD: Modeling solid state drives for holistic system simulation. *IEEE Comput. Arch. Lett.* 17, 1 (Jan. 2018). doi:10.1109/lca.2017.2750658
- [22] Jesung Kim, Jong Min Kim, S H Noh, Sang Lyul Min, and Yookun Cho. 2002. A space-efficient flash translation layer for CompactFlash systems. *IEEE Trans. Consum. Electron.* 48, 2 (May 2002). doi:10.1109/tce.2002.1010143
- [23] J Kim and I Shin. 2016. Multi-thread flash translation layer for multi-core solid state drives. *International Journal of Applied Engineering Research* 11, 2 (March 2016), 1187–1191. <https://pure.seoultech.ac.kr/en/publications/multi-thread-flash-translation-layer-for-multi-core-solid-state-d/>
- [24] Ji-Hoon Kim, Seunghee Han, Kwanghyun Park, Soo-Young Ji, and Joo-Young Kim. 2024. Trinity: In-database near-data machine learning acceleration platform for advanced data analytics. *IEEE Access* 12 (2024). doi:10.1109/access.2023.3347725
- [25] Gunjae Koo, Yunho Oh, Hung-Wei Tseng, Won Woo Ro, and Murali Annavaram. 2023. FLIXR: Embedding Index Into Flash Translation Layer in SSDs. *IEEE Trans. Comput.* 72, 1 (Jan. 2023). doi:10.1109/tc.2022.3154602
- [26] Jaewook Kwak, Sangjin Lee, Kibin Park, Jinwoo Jeong, and Yong Ho Song. 2020. Cosmos+ OpenSSD: Rapid Prototype for Flash Storage Systems. *ACM Trans. Storage* 16, 3 (2020). doi:10.1145/3385073
- [27] Jean J Labrosse. 2007. *μC/OS-II Documentation*. <https://github.com/weston-embedded/uC-OS2>. Accessed: 2026-03-09.
- [28] Gyewon Lee, Jeongyoon Eo, Jangho Seo, Taegeon Um, and Byung-Gon Chun. 2018. High-performance stateful stream processing on solid-state drives. In *Proceedings of the 9th Asia-Pacific Workshop on Systems*. ACM, New York, NY, USA, 1–7. doi:10.1145/3265723.3265739
- [29] Sang-Won Lee, Dong-Joo Park, Tae-Sun Chung, Dong-Ho Lee, Sangwon Park, and Ha-Joo Song. 2007. A log buffer-based flash translation layer using fully-associative sector translation. *ACM Trans. Embed. Comput. Syst.* 6, 3 (July 2007). doi:10.1145/1275986.1275990
- [30] Alberto Lerner and Philippe Bonnet. 2024. *Principles of database and SSDs codesign*. Springer, Berlin, Germany. <https://link.springer.com/book/9783031578762>
- [31] Alberto Lerner, Jaewook Kwak, Sangjin Lee, Kibin Park, Yong Ho Song, and Philippe Cudré-Mauroux. 2020. It takes two: Instrumenting the interaction between in-memory databases and solid-state drives. *CIDR 2020, 10th Conference on Innovative Data Systems Research, Amsterdam, The Netherlands, January 12–15, 2020, Online Proceedings* (2020). <https://folia.unifr.ch/rodoc/330495>
- [32] Huaicheng Li, Martin L Putra, Ronald Shi, Xing Lin, Gregory R Ganger, and Haryadi S Gunawi. 2021. IODA: A Host/Device Co-Design for Strong Predictability Contract on Modern Flash Storage. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*. doi:10.1145/3477132.3483573
- [33] Kiran Kumar Matam, Gunjae Koo, Haipeng Zha, Hung-Wei Tseng, and Murali Annavaram. 2019. GraphSSD: Graph Semantics Aware

- SSD. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*. IEEE. <https://ieeexplore.ieee.org/document/8980325/>
- [34] John M Mellor-Crummey and Michael L Scott. 1991. Algorithms for scalable synchronization on shared-memory multiprocessors. *ACM Transactions on Computer Systems* 9, 1 (Feb. 1991). doi:10.1145/103727.103729
- [35] Kibin Park, Alberto Lerner, Sangjin Lee, Philippe Bonnet, Yong Ho Song, Philippe Cudré-Mauroux, and Jungwook Choi. 2024. BABOL: A Software-Defined NAND Flash Controller. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE. doi:10.1109/micro61859.2024.10869629
- [36] Hireen D. Patel, Ben Lickly, Bruce Huber, and Edward A. Lee. 2008. Predictable Programming on a Precision Timed Architecture. In *Proceedings of the 2008 International Conference on Compilers, Architectures and Synthesis for Embedded Systems (CASES)*. ACM.
- [37] Shuyi Pei, Jing Yang, and Qing Yang. 2019. REGISTOR: A Platform for Unstructured Data Processing Inside SSD Storage. *ACM Transactions on Storage* 15, 1 (April 2019). doi:10.1145/3310149
- [38] Neha Prakriya, Yu Yang, Baharan Mirzasoleiman, Cho-Jui Hsieh, and Jason Cong. 2023. NeSSA: Near-storage data selection for accelerated machine learning training. In *Proceedings of the 15th ACM Workshop on Hot Topics in Storage and File Systems*. ACM, New York, NY, USA. doi:10.1145/3599691.3603404
- [39] Yunhui Qiu, Wenbo Yin, and Lingli Wang. 2022. A high-performance and scalable NVMe controller featuring hardware acceleration. *IEEE Trans. Comput.-aided Des. Integr. Circuits Syst.* 41, 5 (May 2022). doi:10.1109/tcad.2021.3088784
- [40] Luis Cavazos Quero, Young-Sik Lee, and Jin-Soo Kim. 2015. Self-sorting SSD: Producing sorted data inside active SSDs. In *2015 31st Symposium on Mass Storage Systems and Technologies (MSST)*. IEEE. doi:10.1109/msst.2015.7208281
- [41] Tianyu Ren, Yajuan Du, Jinhua Cui, Yina Lv, Qiao Li, and Chun Jason Xue. 2026. Device-level optimization techniques for solid-state drives: A survey. *arXiv [cs.AR]* (Feb. 2026). arXiv:2507.10573 [cs.AR] doi:10.48550/arXiv.2507.10573
- [42] Erik Riedel, Christos Faloutsos, Garth A Gibson, and David Nagle. 2001. Active Disks for Large-Scale Data Processing. *Computer* 34, 6 (June 2001). doi:10.1109/2.928624
- [43] Zhenyuan Ruan, Tong He He, and Jason Cong. 2019. INSIDER: Designing In-Storage Computing System for Emerging High-Performance Drive. In *USENIX Annual Technical Conference*.
- [44] Sudharsan Seshadri, Mark Gahagan, Sundaram Bhaskaran, Trevor Bunker, Arup De, Yanqin Jin, Yang Liu, and Steven Swanson. 2014. Willow: a user-programmable SSD. In *Proceedings of the 11th USENIX conference on Operating Systems Design and Implementation (OSDI'14)*. USENIX Association, USA, 67–80. <https://dl.acm.org/doi/10.5555/2685048.2685055>
- [45] Solidigm. 2025. *SSD Firmware Advisory*. Technical Report. [https://www.solidigmtechnology.com/content/dam/solidigm/en/site/support/support-community/cve-\(security\)/documents/public-security-advisory-v2.pdf](https://www.solidigmtechnology.com/content/dam/solidigm/en/site/support/support-community/cve-(security)/documents/public-security-advisory-v2.pdf)
- [46] Daniel J Sorin, Mark D Hill, and David A Wood. 2011. *A primer on memory consistency and cache coherence*. Springer International Publishing, Cham. doi:10.1007/978-3-031-01733-9
- [47] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and O Mutlu. 2018. MQSim: A framework for enabling realistic studies of modern multi-queue SSD devices. *File Storage Technol* (Feb. 2018). doi:10.5555/3189759.3189765
- [48] Andrew Waterman and Krste Asanović (Eds.). 2019. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. RISC-V Foundation. <https://github.com/riscv/riscv-isa-manual/releases/download/Ratified-IMAFDQC/riscv-spec-20191213.pdf>
- [49] Simon Wegener. 2017. Towards Multicore WCET Analysis. doi:10.4230/OASICS.WCET.2017.7
- [50] Mark Wilkening, Udit Gupta, Samuel Hsia, Caroline Trippel, Carole-Jean Wu, David Brooks, and Gu-Yeon Wei. 2021. RecSSD: near data processing for solid state drive based recommendation inference. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, New York, NY, USA. doi:10.1145/3445814.3446763
- [51] Eugene Yip, Alain Girault, Partha S Roop, and Morteza Biglari-Abhari. 2023. Synchronous deterministic parallel programming for multi-cores with ForeC. *ACM Trans. Program. Lang. Syst.* 45, 2 (June 2023). doi:10.1145/3591594
- [52] J Zhang, M Kwon, M Swift, and M Jung. 2020. Scalable parallel flash firmware for many-core architectures. *18th USENIX Conference on File* (2020). <https://www.usenix.org/conference/fast20/presentation/zhang-jie>
- [53] Jie Zhang, Gieseok Park, Mustafa M Shihab, David Donofrio, John Shalf, and Myoungsoo Jung. 2015. OpenNVM: An open-sourced FPGA-based NVM controller for low level memory characterization. In *2015 33rd IEEE International Conference on Computer Design (ICCD)*. IEEE. doi:10.1109/iccd.2015.7357179
- [54] Xiangqun Zhang, Janki Bhimani, Shuyi Pei, Eunji Lee, Sungjin Lee, Yoon Jae Seong, Eui Jin Kim, Changho Choi, Eyee Hyun Nam, Jongmoo Choi, and Bryan S Kim. 2025. Storage abstractions for SSDs: The past, present, and future. *ACM Trans. Storage* 21, 1 (Feb. 2025). doi:10.1145/3708992
- [55] Jingyu Zhou, Meng Xu, Alexander Shraer, Bala Namasivayam, Alex Miller, Evan Tschannen, Steve Atherton, Andrew J Beamon, Rusty Sears, John Leach, Dave Rosenthal, Xin Dong, Will Wilson, Ben Collins, David Scherer, Alec Grieser, Young Liu, Alvin Moore, Bhaskar Muppana, Xiaoge Su, and Vishesh Yadav. 2021. FoundationDB: A Distributed Unbundled Transactional Key Value Store. In *Proceedings of the 2021 International Conference on Management of Data*. ACM, New York, NY, USA. doi:10.1145/3448016.3457559